Integer LLM with Prolog Knowledge Base

Bidirectional Neural-Symbolic Architecture for Provenanced Generation

Registry: [HOWL-LLM-1-2026]

DOI: 10.5281/zenodo.19528587

Date: March 29 2026

Domain: Machine Learning / Knowledge Representation / Neural-Symbolic Systems

Status: Architecture Specification

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections and one biographical note were edited by the author. All paper content was LLM-generated using Anthropic's Claude 4.5 Sonnet.

I. ABSTRACT

This paper specifies an integrated architecture combining an integer-only neural network with a Prolog knowledge base as a bidirectional substrate. The LLM reads from Prolog and writes to Prolog. Training data enters as Prolog facts with provenance. LLM output becomes Prolog facts after verification. The knowledge base is the persistent memory. The LLM is the pattern engine. Prolog is the verification and storage layer.

Three layers. Layer 1: integer neural network with i32 weights and i16 gradient remainders, zero floating point operations at any stage of training or inference, transcendental functions from precomputed integer pair cache at sub-Planck precision. Layer 2: Prolog knowledge base with typed Terms carrying source, timestamp, confidence, verification level, version, and Quadrium evaluation scores — all integers. Layer 3: bidirectional interface where the LLM reads structured facts from the KB as input context, generates structured Terms as output, submits Terms to Prolog for verification, and verified Terms enter the KB as new facts.

The architecture replaces the context window with a persistent provenanced fact store, replaces RAG with exact predicate matching, replaces BPE tokenization with typed Term-based tokenization, replaces epsilon equality with binary integer equality, and replaces post-hoc hallucination mitigation with structural verification at every generation step.

Existing implementation in Zig provides the working components: integer weight mechanics with complete forward and backward pass (lib.zig), BPE tokenizer (tokenize.zig), training loop with shell transition monitoring (train.zig), autoregressive inference (infer.zig), compilation-based evaluation harness (eval.zig), and Prolog engine with Term/Fact/Rule/KnowledgeBase (prolog.zig). The architecture specification describes the integration of these components into one system.

Prototype: 124M parameter model, Zig, zero floats, Zig standard library parsed into approximately 50,000 Prolog facts, code completion task, compared against float BF16 baseline.

II. WHAT WE ARE BUILDING

One system. Three layers. Each layer handles what it is good at.

The integer neural network handles pattern recognition, fuzzy input comprehension, and creative selection. It maps ambiguous human input to structured representations. It chooses between valid options based on patterns observed in training. It recognizes when a combination of facts is unusual or interesting. These are things neural networks do well.

The Prolog knowledge base handles fact storage, logical verification, consistency enforcement, version filtering, and provenance tracking. It stores what the system knows as explicit, typed, provenanced facts. It checks whether new claims are consistent with existing knowledge. It resolves contradictions structurally. It filters facts by version, confidence, and source. These are things logical systems do well.

The bidirectional interface connects them. Training data flows into Prolog as facts. Facts flow into the LLM as structured context. LLM output flows back into Prolog as proposed facts. Prolog verifies and either accepts (fact enters KB) or rejects (explanation feeds back to LLM for retry). Each cycle adds verified knowledge. Each cycle reduces the space of possible errors.

The LLM is the interface. The knowledge base is the mind. The Prolog engine is the verification layer. They do not compete. They cover each other's weaknesses.

III. THE INTEGER NEURAL NETWORK

III.I Weight Representation

Every learnable parameter in the network is two integers:

```
const Weight = packed struct {

v: i32 = 0,

r: i16 = 0,

};
```

The v field is the weight value. It participates in forward and backward computation. The r field is the remainder — accumulated gradient pressure. It is touched only during the weight update step.

Storage: 6 bytes per parameter. Comparable to FP32 (4 bytes), larger than BF16 (2 bytes). The v field does the computation. The r field preserves gradient signals that float arithmetic destroys.

III.II The Update Rule

The weight update is three integer operations:

```
1. Scale gradient by learning rate:   scaled = -(gradient >> lr_shift)

2. Accumulate in remainder:           r += scaled

3. Transition if threshold reached:

 while r >=  threshold: v += 1, r -= threshold

 while r <= -threshold: v -= 1, r += threshold
```

The learning rate is a bit shift. Shift by 4 divides the gradient by 16. Shift by 8 divides by 256. Higher shift means more gradient samples must accumulate before a transition occurs.

The threshold determines how much accumulated evidence triggers a state change. It is a hyperparameter. Different tasks, different model sizes, and different training regimes may require different thresholds. The correct threshold is whatever produces the best training dynamics for the task. This is an empirical question.

No floating point operation appears in this rule. The weight update is integer addition, integer comparison, and integer subtraction.

III.III Shell Stability

Between transitions, v does not change. The forward pass reads v. The backward pass computes gradients with respect to v. Small gradients accumulate in r without affecting v. Only when accumulated evidence reaches the threshold does v move.

This is inherent noise resistance. Small random gradients add to r but rarely accumulate enough to trigger transitions. Positive and negative noise contributions cancel over time in the remainder. Only persistent, directional gradient signal builds enough pressure to force a transition.

This is the property that dropout, weight decay, and gradient clipping attempt to achieve through external mechanisms. The threshold provides it structurally. The barrier for changing a weight is built into the number format, not added as a regularization hyperparameter.

III.IV The Remainder Is Not Error

In floating point arithmetic, the difference between a computed value and the true value is rounding error — unknown, untracked, irrecoverable.

In integer weight arithmetic, r is not error. It is live data. It is the exact accumulated evidence for or against the next transition. It has a precise integer value. It can be inspected, compared, and tracked. The system does not lose information. It defers acting on information until the evidence is sufficient.

III.V Convergence

In float training, "converged" means the loss stopped decreasing noticeably. Weights are still jittering at the float precision floor. There is no true equilibrium.

In integer training, convergence means all |r| values have stabilized below the transition threshold. Every weight is in its state. No transitions are occurring. This is a discrete equilibrium — verifiable by a single pass over all weights checking |r| < threshold.

III.VI Forward Pass

The core operation is integer matrix-vector multiply:

```
pub fn matmul_vec_weight(input: []const i32, w: *const WeightMatrix, outpu

for (0..cols) |j| {

    var acc: i64 = 0;

    for (0..rows) |i| {

        acc += @as(i64, input[i]) * @as(i64, w.weights[i * cols + j].v);

    }

    output[j] = @intCast(std.math.clamp(acc >> OCTAVE_SHIFT, -2147483647,

}

}
```

Multiply is i32 $\times$ i32 $\rightarrow$ i64. Accumulation is i64 addition. The shift normalizes back to the working precision. All operations exact until the final shift, which discards bits below the working precision — controlled and explicit, unlike float rounding which is silent and untracked.

ReLU is naturally integer: if x < 0, output 0; otherwise output x. No approximation.

III.VII Transcendental Functions from MATH-2 Cache

GELU, softmax, and any activation function requiring transcendental values are implemented via precomputed integer lookup tables.

[HOWL-MATH-2-2026] established that every transcendental constant appearing in physics and mathematics can be represented as an exact integer pair (p, q) at 100 decimal digits — a precision exceeding the Planck length by 65 orders of magnitude. The 17 constants confirmed include π , e, ln(2), and every value needed for activation function computation.

The lookup tables are computed once at program startup:

1. Compute the transcendental function at each table entry using MATH-2 integer pairs in exact rational arithmetic.
2. Store the results as integer arrays at the working precision.
3. At inference, look up the nearest entry. Between entries, linear interpolation in integer arithmetic: one multiply, one add, one shift.

No float appears in the transcendental computation. The lookup tables are exact at the working precision. The interpolation is explicit and bounded. The approximation is in the interpolation between table entries, not in the number format.

III.VIII Backward Pass

The gradient of every operation in the forward pass is computable in integer arithmetic.

The gradient of integer matmul Y = W $\times$ X with respect to W is X^T $\times$ dY — another integer matmul. The gradient with respect to X is W^T $\times$ dY — also an integer matmul.

The gradient of integer ReLU is 1 where pre-activation was positive, 0 where negative. Already integer.

The gradient of integer softmax-cross-entropy with respect to logits is probs[i] - target_indicator[i], where probabilities are integers summing to a fixed scale (1024 in the current implementation). The gradient is integer.

Every gradient is an integer computed by integer operations on integer values. No approximation is introduced by the backward pass.

III.IX What Integer Arithmetic Gives You

Exact gradient accumulation. A gradient of 1 accumulated over 1000 steps produces 1000. Not 999.97. The signal that would vanish below the float precision floor survives in the remainder and contributes when sufficient evidence has accumulated.

Determinism. Same input, same weights, same output. Every time. Integer arithmetic is associative and commutative. The result does not depend on thread scheduling, reduction order, or hardware implementation. Reproducible benchmarks. Debuggable behavior. Cacheable computation.

Binary equality. Two values are equal or they are not. No epsilon tolerance. No "close enough." If two computations should produce the same result, they do produce the same result. If they do not, the difference is a real difference, not a rounding artifact.

Structural interpretability. A weight with v=0 and r=0 is dead — prune without loss. A weight with stable v and small |r| has converged. A weight with large |r| oscillating around zero is contested — training may not be complete for this parameter. A weight with steadily climbing |r| is accumulating evidence toward a transition. The structure is readable. No probing experiments required. No ablation studies. The weight tells you what it is.

IV. THE PROLOG KNOWLEDGE BASE

IV.I The Term

The fundamental data carrier is the Term. Every piece of information in the system is a Term or a structure of Terms.

```
pub const TermType = enum(i32) {

atom,           // literal value: "max", "allocator"

variable,       // unification variable: X, Target

number,         // integer value

list,           // list of terms

vector2,        // spatial point (integer coordinates)

rectangle,      // spatial region (integer coordinates)

entity,         // entity reference (index)

// code-specific

name,           // identifier

kind,           // category: fn, struct, noun, verb

type_ref,       // type: i32, bool, []u8

keyword,        // control: if, while, return

operator,       // arithmetic/logic: + - * ==
```

```
// logic-specific

predicate,      // predicate name

rule,           // rule reference

fact,           // fact assertion

};
```

Each Term carries its grammatical or semantic role in its type field. A Term of type `.type_ref` with value `"i32"` is structurally different from a Term of type `.name` with value `"i32"`. The type system prevents category confusion that plagues flat token representations.

The Term struct from prolog.zig carries spatial data (vectors, rectangles), entity references (indices), and lists. For the LLM architecture, the Term is extended with provenance and evaluation fields:

```
pub const Term = struct {

type: TermType = .atom,

atom: Text = Text.initEmpty(),

variable: Text = Text.initEmpty(),

number: i32 = 0,

index: i32 = -1,

vec2: Vector2Int = .{},

rect: RectInt = .{},

list: []Term = &[_]Term{},

// provenance

source: Source = .training_corpus,

timestamp: i32 = 0,

confidence: Confidence = .speculative,
```

```
verification: Verification = .none,
source_id: u64 = 0,
offset: u64 = 0,
length: u32 = 0,
version: u32 = 0,
supersedes: ?u64 = null,
// quadrium evaluation
score_l: u8 = 0,
score_m: u8 = 0,
score_e: u8 = 0,
score_u: u8 = 0,
};
pub const Source = enum(i32) {
user_prompt,
user_correction,
llm_inference,
prolog_derivation,
training_corpus,
runtime_ingestion,
};
pub const Confidence = enum(i32) {
stated,
inferred,
```

```
derived,

speculative,

};

pub const Verification = enum(i32) {

high,

medium,

low,

none,

};
```

All integers. No floats. Every field exact. Every fact traceable from output to source bytes.

IV.II Facts and Rules

A Fact is a predicate with typed Term arguments — identical to prolog.zig:

```
pub const Fact = struct {

predicate: Text,

args: []Term,

};
```

A Rule is a head predicate with body facts — if all body facts unify, the head is true:

```
pub const Rule = struct {

head: Text,

body: []Fact,

};
```

FactSets and RuleSets compose into KnowledgeBases — identical to the game engine architecture. Each domain has its own FactSet (parsed source material) and RuleSet (domain patterns). They combine into the unified KB that serves the LLM.

IV.III Multi-Dimensional Indexing

Following [HOWL-INFO-1-2026], facts are not flat claims. They carry their full context.

Temporal specificity. A fact about Zig 0.14 API coexists with a fact about Zig 0.15 API. Both are in the KB. Neither contradicts the other because they carry different version tags.

Source tracking. The user (authoritative for intent), the training corpus (confidence depends on source quality), LLM inference (speculative until verified), Prolog derivation (derived confidence based on input facts).

Confidence grading. Not a float probability. An explicit integer category: stated, inferred, derived, speculative.

Verification depth. Can this be independently checked? High (verifiable against source at stored offset), medium (reasonable inference from verified sources), low (weak evidence), none.

Supersession. When a fact is corrected, the old fact is not deleted. It is superseded. Both exist in the KB. Queries return the current unsuperseded version. History preserved for audit.

IV.IV Version Filtering

Every fact derived from versioned source material carries its version:

```
valid_fact(Fact) :-

provenance(Fact, SourceId),

source_version(SourceId, Version),

query_version(RequestedVersion),

Version == RequestedVersion.
```

This is a hard filter. Facts from version 14 do not enter the search space when version 15 is requested. They are not "less likely." They are excluded. The query never encounters them.

IV.V Contradiction Detection

```
contradiction(Fact1, Fact2) :-

same_predicate(Fact1, Fact2),

same_entity(Fact1, Fact2),
```

```
same_version(Fact1, Fact2),

different_value(Fact1, Fact2),

neither_supersedes(Fact1, Fact2).
```

When a contradiction is detected, the system does not silently average across both facts. It flags the contradiction, reports the sources and confidence levels of both facts, and requests resolution — either from Prolog rules (prefer higher confidence) or from the user.

IV.VI Quadrium Evaluation

Following [HOWL-SOPH-2-2026], every fact is scored on four independent axes:

L — Logical validity (0-100). Is the claim internally consistent? Does it follow from its premises?

M — Mathematical coherence (0-100). Are the quantities computable? Are the types compatible? Does the arithmetic compile?

E — Empirical anchoring (0-100). Does the claim trace to verifiable sources? How many independent sources support it?

U — Demonstrated utility (0-100). Does the claim produce functional output? Does the code compile? Does the system work?

Four integer scores. Independent. U does not compensate for M. High utility does not excuse absent mathematical compilation. A fact with L=90, M=0, E=85, U=99 is the Standard Model's profile — and the system reports all four scores, making the M-failure visible alongside the utility.

IV.VII The KB Replaces the Context Window

The context window is a fixed-size token buffer with no metadata, no provenance, no consistency enforcement, and no persistence. When it fills, old tokens are discarded permanently.

The KB has no size limit — bounded only by available storage. No positional degradation — fact number 1 is matched with the same mechanism as fact number 100,000. No attention competition — queries retrieve only matching facts, irrelevant facts are structurally excluded. Exact matching — integer predicate matching is exact, no "similar enough" threshold. Consistency enforcement — Prolog rules check for contradictions at any time. Persistence — facts survive across sessions.

Sessions are views into the KB. Multiple sessions share one KB with independent context filters. A session that pauses and resumes months later finds its full state waiting. Nothing forgotten. Nothing degraded.

IV.VIII Memory Management

Three temperature tiers based on access patterns:

Hot: In RAM. Recently accessed. Query response in nanoseconds.

Warm: Memory-mapped. Paged in on access. Response in microseconds.

Cold: On disk. Loaded on demand. Response in milliseconds.

LRU eviction moves cold facts to disk. Nothing is deleted. An evicted fact exists on disk with full provenance and metadata. If a query matches an evicted fact, the fact is loaded, served, and promoted.

Eviction is intelligent:

```
protect(Fact) :- source(Fact, user_prompt).

protect(Fact) :- source(Fact, user_correction), age(Fact, A), A < 1000.

prefer_eviction(Fact) :- confidence(Fact, speculative).
```

User instructions are never evicted. Recent corrections are protected. Speculative facts are evicted first. The context window cannot do this — it discards tokens based solely on position.

V. THE BIDIRECTIONAL INTERFACE

V.I Training Data → Prolog

Training data is parsed into Prolog facts with provenance. A domain-specific parser produces Terms:

Zig source → Terms:

```
pub fn max(a: i32, b: i32) i32 { return @max(a, b); }

→ Fact{ predicate="function", args=[

Term{ .kind, "fn" },

Term{ .name, "max" },

Term{ .type_ref, "i32", source=.training_corpus,

    source_id=hash("std/math.zig"), offset=4821, version=15 },
```

```
Term{ .type_ref, "i32" },

Term{ .type_ref, "i32" },  // return type

  ]}
```

Each fact carries the source file hash, byte offset, and version. The provenance chain is complete from any fact back to the exact bytes in the exact source file.

V.II Prolog → LLM Context

Before each generation step, the LLM receives relevant facts from the KB. Not a raw text context window. Structured facts selected by predicate matching, filtered by version, sorted by confidence.

The LLM's input is a marshaled set of facts relevant to the current query. Facts that match the query predicate enter the context. Facts from the wrong version are excluded. Facts with higher confidence appear first. The LLM processes structured, typed, provenanced data — not a linear sequence of undifferentiated tokens.

V.III LLM → Prolog

The LLM generates Terms, not raw text. Each Term carries its type. The Terms are submitted to Prolog for verification:

```
verify(Term) :-

type_check(Term),

consistency_check(Term),

version_check(Term).

type_check(Term) :-

term_type(Term, type_ref),

known_type(Term, version=V),

query_version(V).

consistency_check(Term) :-

not(contradicts(Term, AnyExistingFact)).
```

Verified Terms become facts in the KB with source=llm_inference and confidence=inferred. Rejected Terms generate explanations:

```
Rejection{ term=Term, reason="@intCast requires error handling in Zig 0.15",
    rule=zig_015_error_handling, existing_fact=Fact{...} }
```

The rejection reason enters the LLM's context for the next generation step. The LLM retries with the constraint information. The cycle continues until verification passes or a retry limit is reached.

V.IV The Alternating Pipeline

The complete generation pipeline for a single request:

```
1. User prompt arrives.
2. LLM parses prompt to structured Terms (fuzzy comprehension).
3. Prolog verifies understanding against KB.
4. Prolog decomposes goal into steps with requirements.
5. For each step:
   a. LLM generates Terms for this step.
   b. Prolog verifies Terms against KB.
   c. If verified: Terms become facts. Advance.
   d. If rejected: explanation feeds back. LLM retries.
6. All steps verified. Emit output from verified Terms.
```

The emission step is deterministic. Each verified Term has exactly one text representation. The emitter converts Terms to text with appropriate formatting. The LLM does not write the final output. The emitter writes it from verified Terms.

VI. THE EXECUTION CONTROLLERS

Three controllers from the INFO series, implemented as Prolog rules operating on the KB.

VI.I Scales Method — Materiality Gate

Following [HOWL-INFO-2-2026], before spending cycles on verification or generation, the system checks materiality:

```
material(Claim) :- changes_outcome(Claim), scope(Claim, P), P > 5.

process(Claim) :- material(Claim).

tag_only(Claim) :- not(material(Claim)).
```

A claim is material if it changes the outcome and affects a non-negligible percentage of cases. Non-material claims are tagged but not deeply processed. This prevents the system from spending compute on irrelevant verification work.

Scope quantification is integer:

```
severity(Claim, negligible)    :- scope(Claim, P), P < 5.

severity(Claim, minor)         :- scope(Claim, P), P >= 5,  P < 20.

severity(Claim, significant)   :- scope(Claim, P), P >= 20, P < 50.

severity(Claim, major)         :- scope(Claim, P), P >= 50, P < 80.

severity(Claim, critical)      :- scope(Claim, P), P >= 80.
```

Integer percentages. Exact comparison. No float probabilities.

VI.II Pseudo-Socratic Method — Sequencing and State Tracking

Following [HOWL-INFO-3-2026], the execution controller determines how many LLM↔Prolog steps to take and what each step focuses on.

State assessment before each step:

```
comprehension(Topic, solid)           :- all_prereqs_met(Topic), demonstrate

comprehension(Topic, gap)             :- missing_prereq(Topic, _).

comprehension(Topic, confused)        :- contradictory_signals(Topic).

comprehension(Topic, not_introduced)  :- not(encountered(Topic)).
```

Action selection:

```
next(advance, Step)     :- all_previous_verified(Step), requirements_met(Ste
```

```
next(retry, Step)       :- rejected(Step, _), retries(Step, N), N < max_retr
next(backfill, Prereq)  :- requires(Step, Concept), comprehension(Concept, g
next(complete)          :- all_verified, goal_met.
```

Two modes. **Convergent:** goal known, steps decomposed, each verified before proceeding. "Write a function that does X." **Divergent:** goal open-ended, options evaluated by utility, best path taken. "Explore approaches to this design problem."

Multi-prompt state tracking. The PS tracker persists across sessions. A paused series — tutorial, project, investigation — resumes exactly where it left off. State is Prolog facts in the KB:

```
series(tutorial_allocators, status=paused, step=3, timestamp=T).
completed(tutorial_allocators, step_1, timestamp=T1).
completed(tutorial_allocators, step_2, timestamp=T2).
comprehension(allocators_basic, solid, timestamp=T2).
comprehension(arena_pattern, not_introduced, timestamp=T2).
```

When the user returns — "let's go back to the allocator tutorial" — the PS tracker retrieves the series state, knows steps 1-2 are complete, knows basic allocators are solid, knows arena pattern hasn't been introduced, and picks up exactly where it left off.

VI.III Information Locality — Trust Hierarchy

Following [HOWL-INFO-4-2026], when the system has a unique set of information and accuracy is critical, only local data is valid.

```
trust(Fact, high)       :- source(Fact, user_prompt), accuracy_critical.
trust(Fact, high)       :- source(Fact, training_corpus), verification(Fact
trust(Fact, medium)     :- source(Fact, prolog_derivation), accuracy_critic
trust(Fact, low)        :- source(Fact, llm_inference), accuracy_critical.
trust(Fact, acceptable) :- source(Fact, llm_inference), not(accuracy_critic
```

When accuracy matters — generating code, stating facts, making claims — the system sources from the KB. The LLM's statistical patterns are non-local data. They are useful

for fuzzy comprehension and creative selection. They are not valid for accuracy-critical decisions. The trust hierarchy encodes this directly.

VII. DOMAIN EATING

Adding knowledge domains without retraining the neural network.

VII.I The Process

1. Obtain source material for the domain.
2. Write a domain-specific parser that produces Terms with provenance.
3. Write Prolog rules encoding the domain's valid patterns and constraints.
4. Run the parser on the source material. Facts enter the KB.
5. Validate consistency — check for contradictions among ingested facts.
6. Domain is live. No retraining.

The neural network's weights do not change. The KB grows. New facts are immediately queryable. New rules are immediately applicable.

VII.II Domain-Specific Parsers

Each domain has a parser producing the universal Term format:

Zig source → Terms. Uses Zig's own tokenizer. Emits Terms with types .keyword, .name, .type_ref, .operator. Provenance is source file hash and byte offset.

C headers → Terms. Uses a C tokenizer. Emits function signatures, type definitions, macro definitions. Provenance is header file hash and byte offset.

English text → Terms. Word-level tokenization with part-of-speech classification. Emits Terms with types .kind (noun, verb, adjective), .name. Provenance is document hash and position.

Any new domain → Terms. Write a parser. The output format is always typed Terms with provenance. The rest of the system does not change.

VII.III Domain Rules

Each domain has Prolog rules encoding valid patterns:

```
% Zig: a function call is valid if the function exists at the requested ve

valid_call(Fn, Args) :-
```

```
function(Fn, source=S),

source_version(S, V),

query_version(V),

match_params(Fn, Args).

% Zig: error handling required for fallible functions

requires_error_handling(Call) :-

valid_call(Call, _),

fallible(Call).
```

Rules are small — typically a few hundred per domain. They are explicit, inspectable, and correctable. If a rule is wrong, it is identified by the provenance of the incorrect output and fixed directly.

VII.IV Cross-Domain Queries

All domains produce the same Term format. All facts live in the same KB. Cross-domain queries work through shared predicates:

```
% A Zig function wraps a C function

wraps(ZigFn, CFn) :-

calls_via_cimport(ZigFn, CFn, source=zig_source),

function(CFn, source=c_header).
```

No special cross-domain mechanism. Prolog unification over shared predicates connects facts across domains naturally.

VIII. WHAT THE LLM DOES AND DOES NOT DO

VIII.I The LLM Does

Fuzzy input comprehension. Mapping misspelled, ambiguous, colloquial human input to structured Terms. “make me a functon that takes two numbrs” → Terms with correct types and atoms. This is pattern matching — the LLM’s core strength.

Creative option selection. When Prolog presents multiple valid options for a generation step, the LLM selects the most appropriate based on patterns in training data. “What’s

the most idiomatic Zig pattern for this?" is a question the LLM can answer well.

Anomaly recognition. Noticing that a combination of facts, while individually valid, is unusual or interesting. "This set of constraints is similar to problem X, which was solved by approach Y." Cross-pattern recognition is a neural network strength.

VIII.II The LLM Does Not

Write final output text. The emitter writes output from verified Terms. The LLM never produces the final text.

Enforce constraints. Prolog enforces constraints. The LLM proposes. Prolog disposes.

Track state. The KB tracks state. The LLM does not maintain a mental model of the conversation. The KB does, as explicit facts.

Verify its own output. The LLM's output is always verified by Prolog before it affects anything. Self-verification is a known failure mode of neural networks. External verification by a logical system eliminates it.

Remember across turns. The KB remembers. The LLM processes the current step's context, which includes relevant KB facts marshaled by Prolog. The LLM does not need long-range memory because the KB provides it.

VIII.III Implications for Model Size

Because the LLM handles only comprehension and selection — not knowledge storage, constraint enforcement, state tracking, or output generation — it can be substantially smaller than a general-purpose LLM.

A 100M-1B parameter model may suffice because knowledge is in the KB, not the weights. Grammar and syntax are in the Prolog rules, not learned from data. Provenance is in the fact metadata, not encoded implicitly. Consistency is enforced by Prolog, not hoped for from statistics. The model needs only enough capacity to map fuzzy input to structured Terms and to select between options based on training patterns.

IX. WHAT REPLACES WHAT

Current Architecture	This Architecture
Float weights (BF16/FP32)	Integer weights (i32 + i16 remainder)
Context window (fixed token buffer)	Persistent provenanced KB
RAG (approximate vector similarity)	Exact predicate matching with provenance
BPE tokenization (arbitrary byte-pair fragments)	Term-based tokenization (typed, structured)
Epsilon equality	Binary integer equality
Non-deterministic inference	Deterministic inference
Hallucination mitigated post-hoc	Verification at every generation step

Current Architecture	This Architecture
Knowledge dissolved in weights	Knowledge explicit in KB, patterns in weights
Session-bounded memory	Persistent memory across sessions
RLHF for quality	Prolog verification for correctness
Fixed context degrades with length	KB grows richer with use, no degradation
Version confusion (mixed training data)	Hard version filtering
Silent contradictions	Structural contradiction detection

X. THE PROTOTYPE

X.I Specification

Model: 124M parameters. GPT-2 small equivalent architecture. Zig. Zero floats.

Weights: i32 value + i16 remainder. Threshold determined empirically during development.

Knowledge base: prolog.zig extended with provenance and Quadrium fields. Zig standard library parsed into approximately 50,000 facts with full provenance — file hash, byte offset, version.

Tokenization: Term-based. Zig source parsed by Zig's own tokenizer into typed Terms. Each token carries its grammatical role.

Task: Code completion on Zig source. Chosen because code has unambiguous correctness — generated code compiles or it doesn't, passes tests or it doesn't.

Pipeline:

1. Parse Zig stdlib → Prolog facts with provenance.
2. Train integer LLM on fact-structured data.
3. At inference: user prompt → parse to Terms → query KB for relevant facts → marshal as LLM context → LLM generates Terms → Prolog verifies against KB → verified Terms emitted as code → code compiled → pass/fail.

Comparison: Float BF16 baseline, same architecture, same data, no Prolog verification.

X.II Metrics

Overall accuracy. pass@1 on code completion benchmarks.

Long-tail accuracy. Rare API calls, uncommon patterns, edge-case syntax.

Hallucination rate. Calls to nonexistent functions, references to nonexistent types, use of wrong-version API.

Consistency. Variance across inference runs. Must be zero for integer architecture.

Training convergence. Loss curves compared between integer and float.

Session coherence. Quality of output at turn 1000 versus turn 1.

Domain eating. Adding a new library's facts to the KB without retraining, measuring immediate completion accuracy for that library.

XI. SUCCESS CRITERIA

1. **Convergence.** Integer training converges to equivalent or lower loss than float baseline.
2. **Long-tail accuracy.** Measurable improvement on rare API calls and uncommon patterns.
3. **Hallucination reduction.** Fewer calls to nonexistent functions than float baseline without Prolog.
4. **Determinism.** Zero variance across inference runs.
5. **Session stability.** Turn 1000 as coherent as turn 1.
6. **Domain eating.** Adding new library facts produces correct completions without retraining.

Any one justifies further development. All six together constitute a new architecture class.

XII. FALSIFICATION CRITERIA

F1. If integer training does not converge on code completion at 124M parameters with any threshold setting, integer neural networks are insufficient for this task at this scale.

F2. If Prolog verification overhead makes generation more than $10\times$ slower than unconstrained float generation for equivalent output length, the alternating pipeline is impractical.

F3. If the KB degrades in consistency over extended use — producing contradictory outputs at turn 10,000 that it would not produce at turn 100 — the consistency guarantees do not hold at scale.

F4. If domain eating (parser + rules + validation) takes longer than fine-tuning the float LLM on equivalent data, the KB approach has no efficiency advantage for adding knowledge.

F5. If the hallucination rate does not decrease relative to the float baseline without Prolog, the verification architecture adds no value for correctness.

F6. If Term-based tokenization produces worse language modeling performance than BPE at the same model size and data, the structural information in Terms is not useful for learning.

Each criterion specifies a concrete test. The paper stands until a specific test demonstrates a specific failure.

XIII. REFERENCES

1. [HOWL-MATH-2-2026] Integer-Pair Representations of Transcendental Constants at Sub-Planck Precision.
2. [HOWL-INFO-1-2026] Multi-Dimensional Information Indexing.
3. [HOWL-INFO-2-2026] The Scales Method.
4. [HOWL-INFO-3-2026] The Pseudo-Socratic Method.
5. [HOWL-INFO-4-2026] Howland's Axiom of Information Locality.
6. [HOWL-SOPH-2-2026] The Quadrium: Truth and Utility as Independent Measurements.

Appendix A: Architecture Component Summary

Table A.1 — Three-Layer Architecture

Layer	Component	Handles	Does Not Handle
1	Integer Neural Network	Fuzzy comprehension, creative selection, pattern recognition	Knowledge storage, constraint enforcement, state tracking, output generation
2	Prolog Knowledge Base	Fact storage, verification, consistency, version filtering, provenance	Pattern matching, ambiguity resolution, creative choice
3	Bidirectional Interface	Marshaling facts to LLM context, submitting Terms for verification, retry with rejection reasons	— (connects Layers 1 and 2)

Table A.2 — Data Flow Through Layers

Stage	Direction	Data	Format
Training ingestion	Source → KB	Parsed source material	Terms with provenance
Context marshaling	KB → LLM	Relevant facts for current query	Structured Term arrays
Generation	LLM → Interface	Proposed new Terms	Typed Terms
Verification	Interface → KB	Terms submitted for checking	Prolog queries
Acceptance	KB → KB	Verified Terms become facts	Facts with source=llm inference
Rejection	KB → LLM	Rejection reason + violated rule	Structured explanation
Emission	KB → Output	Verified Terms assembled	Deterministic text

---

Appendix B: Weight Mechanics

Table B.1 — Weight Storage Comparison

Format	Components	Bits	Bytes	Equality	Deterministic
BF16	1 float (sign + exp + mantissa)	16	2	Epsilon	No
FP32	1 float (sign + exp + mantissa)	32	4	Epsilon	No
FP64	1 float (sign + exp + mantissa)	64	8	Epsilon	No
Integer (this architecture)	i32 value + i16 remainder	48	6	Binary exact	Yes

Table B.2 — Model-Scale RAM Requirements

Model Size	BF16	FP32	Integer (6 bytes)	Training (~5 × weights)
124M (prototype)	0.25 GB	0.50 GB	0.74 GB	3.7 GB
350M	0.70 GB	1.40 GB	2.10 GB	10.5 GB
1.3B	2.6 GB	5.2 GB	7.8 GB	39 GB
7B	14 GB	28 GB	42 GB	210 GB

Table B.3 — Shell Transition Example

Step	Gradient	r Before	r After	v Change	State
1	+7	0	7	0	Pressure building
2	+5	7	12	0	Pressure building
3	+4	12	16	0	Accumulating
4	+3	16	19	0	Accumulating
5	+8	19	27	0	Near threshold
6	+6	27	33	+1	**Transition** (r resets to 33-T)
7	-4	33-T	33-T-4	0	Reverse pressure
8	-9	—	—	0	Pressure retreating
9	+15	—	—	0	Direction reversed
10	+30	—	—	+1	**Transition**

Note: T = threshold (empirical hyperparameter). Table illustrates the mechanics with any threshold. The specific value of T is determined during development.

Table B.4 — Weight Diagnostics

v	r	Interpretation	Action
0	0	Dead weight — no value, no pressure	Prune without loss
V	0	Stable — converged, no residual pressure	None needed
V	small	r	
V	large	r	steady
V		r	oscillating
V		r	at threshold-1

Table B.5 — Convergence Diagnostics

Metric	Meaning	Converged Value	Measurement
Transition rate	% of weights changing v per step	0%	Count v changes per batch
Mean	r		Average remainder pressure
Max	r		Highest pressure in network
r variance	Spread of remainder pressure	Low, stable	Variance of r values
r directional bias	Net positive vs negative r	Near zero	Sum of r across layer

---

Appendix C: Prolog Knowledge Base

Table C.1 — Term Types

TermType	Category	Purpose	Examples
atom	General	Literal value	“max”, “allocator”, “dog”
variable	Logic	Unification variable	X, Target, Result
number	Value	Integer value	42, -7, 0
list	Structure	List of terms	[X, Y, Z]
vector2	Spatial	Integer point	(100, 250)
rectangle	Spatial	Integer region	(0, 0, 800, 600)
entity	Reference	Entity index	entity 5, entity 12
name	Code	Identifier	max, allocator, buf
kind	Code	Category keyword	fn, struct, noun, verb
type ref	Code	Type reference	i32, bool, []u8
keyword	Code	Control keyword	if, while, return, try
operator	Code	Arithmetic/logic	+, -, *, ==, >
predicate	Logic	Predicate name	function, param, returns
rule	Logic	Rule reference	valid call, type check
fact	Logic	Fact assertion	function(max, [i32, i32], i32)

Table C.2 — Source Types

Source	Meaning	Default Confidence	Default Verification	Trust (accuracy critical)
user prompt	User directly stated	stated	high	high
user correction	User corrected previous fact	stated	high	high
llm inference	Neural network inferred	inferred	low	low
prolog derivation	Logically derived from other facts	derived	depends on inputs	medium
training corpus	Extracted from training data	inferred	medium	medium
runtime ingestion	Added from user-provided source	stated	medium-high	medium-high

Table C.3 — Confidence Levels

Level	Meaning	Source Examples	Prolog Priority
stated	Directly asserted by authoritative source	User prompt, verified documentation	Highest
inferred	Concluded by pattern matching	LLM output, statistical pattern	Low until verified
derived	Logically derived from other facts	Prolog rule conclusion	Depends on input facts
speculative	Uncertain, needs verification	LLM guess, weak pattern	Lowest

Table C.4 — Verification Levels

Level	Meaning	Examples	Eviction Priority
high	Verifiable against source at stored offset	Stdlib function signature at known byte offset	Protected
medium	Reasonable inference from verified sources	Type compatibility derived from verified signatures	Normal

Level	Meaning	Examples	Eviction Priority
low	Weak evidence or single unverified source	LLM inference from sparse training data	Preferred for eviction
none	No verification possible	Speculative claim with no source	First to evict

Table C.5 — Fact Metadata Fields

Field	Type	Purpose	Example
predicate	Text	What this fact asserts	“function”, “param”, “returns”
args	[]Term	Structured arguments	[name(“max”), type ref(“i32”)]
source	Source (enum i32)	Who/what produced this fact	.training corpus
timestamp	i32	When (turn number or absolute)	1, 2, 47
confidence	Confidence (enum i32)	How certain	.stated
verification	Verification (enum i32)	Can it be checked	.high
source id	u64	Hash of source file	hash(“std/math.zig”)
offset	u64	Byte position in source	4821
length	u32	Span of source material	47
version	u32	Source material version	15
supersedes	?u64	Fact ID this replaces	fact 247 or null
score l	u8	Logical validity 0-100	90
score m	u8	Mathematical coherence 0-100	85
score e	u8	Empirical anchoring 0-100	92
score u	u8	Demonstrated utility 0-100	99

Table C.6 — KB Operations

Operation	Input	Output	Side Effect
Assert	Fact with provenance	Fact ID	Fact enters KB
Query	Predicate + args + version filter	Matching facts sorted by confidence	None (read-only)
Supersede	New fact + old fact ID	New fact ID	Old fact marked superseded
Contradict	(automatic)	Contradiction report	Flagged for resolution
Evict	LRU threshold	Evicted fact IDs	Facts moved to disk
Restore	Fact ID (on disk)	Restored fact	Fact loaded to RAM

Table C.7 — KB vs Context Window vs RAG

Property	Context Window	RAG	Knowledge Base
Structure	Linear token buffer	Text chunks + vector DB	Indexed typed facts
Size	Fixed (4K-128K tokens)	Limited by vector DB	Unbounded (RAM + disk)
Persistence	Per-session only	Persistent store, per-query retrieval	Persistent, accumulating
Metadata	None	None	Full (source, time, confidence, version)
Matching	Attention (float, approximate)	Vector similarity (float, approximate)	Predicate unification (integer, exact)
Consistency	None	None	Prolog contradiction detection
Version filtering	None	None	Hard filter by version
Degradation	Progressive with length	None (but no accumulation)	None
Information loss	Permanent when truncated	None (but retrieval is approximate)	Never (eviction to disk, not deletion)
Provenance	None	None	Full chain to source bytes
Cross-session	Lost	Available but not accumulated	Accumulated, persistent

Appendix D: Execution Controllers

Table D.1 — Scales Method Severity Classification

Scope (% affected)	Severity	System Response
< 5%	Negligible	Tag only, no deep processing
5-19%	Minor	Process if resources available
20-49%	Significant	Process with normal priority
50-79%	Major	Process with high priority
≥ 80%	Critical	Process immediately

Table D.2 — Pseudo-Socratic State Assessment

Comprehension State	Meaning	Next Action
solid	All prerequisites met, demonstrated understanding	Advance to next step
gap	Missing prerequisite identified	Backfill the prerequisite
confused	Contradictory signals from user	Clarify before proceeding
not introduced	Topic not yet encountered	Defer until prerequisites solid

Table D.3 — Pseudo-Socratic Modes

Mode	Goal Status	Step Selection	Termination
Convergent	Known goal	Decompose into steps, verify each	All steps verified + goal met
Divergent	Open-ended	Evaluate options by utility, take best	Sufficient exploration + satisfactory outcome

Table D.4 — Information Locality Trust Hierarchy

Source	Accuracy Critical	Trust Level	Use For
user prompt	Yes	High	User intent, stated requirements
training corpus (high verification)	Yes	High	Verified API signatures, documented behavior
prolog derivation	Yes	Medium	Derived conclusions from verified inputs
llm inference	Yes	Low	Do not use alone — verify first
llm inference	No	Acceptable	Fuzzy comprehension, creative selection, brainstorming

Table D.5 — Controller Integration

Controller	Source	Implements	Operates On	Prolog Rule Pattern
Scales Method	[HOWL-INFO-2-2026]	Materiality gate	Every claim before processing	material(Claim) :- changes outcome, scope > 5
Pseudo-Socratic	[HOWL-INFO-3-2026]	Sequencing + state tracking	Generation pipeline steps	next(Action, Step) :- state assessment
Information Locality	[HOWL-INFO-4-2026]	Trust hierarchy	Every fact retrieval	trust(Fact, Level) :- source + accuracy criticality
Quadrium	[HOWL-SOPH-2-2026]	Four-axis evaluation	Every fact in KB	evaluation(Fact, l, m, e, u)

Appendix E: Quadrium Evaluation

Table E.1 — Four Axes

Axis	Measures	Range	Independence
L — Logical validity	Internal consistency, non-contradiction, valid derivation	0-100 (u8)	Does not compensate for M, E, or U
M — Mathematical coherence	Quantities computable, types compatible, arithmetic compiles	0-100 (u8)	Does not compensate for L, E, or U
E — Empirical anchoring	Traceable to verifiable sources, independent attestation	0-100 (u8)	Does not compensate for L, M, or U
U — Demonstrated utility	Functional output, code compiles, system works	0-100 (u8)	Does not compensate for L, M, or E

Table E.2 — Failure Mode Taxonomy

Pattern	L	M	E	U	Name	Example
High L+M, Low E+U	High	High	Low	Low	Ivory tower	String theory
High L+E+U, Low M	High	Low	High	High	Utility trap	Standard Model at 19 insertion points
High E, Low L+M	Low	Low	High	Varies	Empiricist trap	Correlation without causation
High L+M, Low E	High	High	Low	Varies	Rationalist trap	Unfalsifiable logical construc-tions
Low all	Low	Low	Low	Low	Dead frame-work	Killed by falsifica-tion
High all	High	High	High	High	Knowledge	Confirmed, compiled, anchored, working

Table E.3 — Quadrium Storage

Field	Type	Bytes	Total for 4 axes
score l	u8	1	—
score m	u8	1	—
score e	u8	1	—
score u	u8	1	—
Total	—	—	**4 bytes per fact**

Appendix F: Domain Eating

Table F.1 — Domain Addition Process

Step	Input	Output	Who/What Does It
1. Obtain source	Domain documentation, source code, text	Raw material	Human or automated retrieval
2. Write parser	Raw material format specification	Parser producing Terms with provenance	Human developer
3. Write rules	Domain patterns and constraints	Prolog rules for domain validation	Human domain expert or LLM-assisted
4. Parse	Raw material + parser	Facts in KB with provenance	Automated
5. Validate	Ingested facts	Consistency report	Prolog contradiction detection
6. Live	—	Domain queryable	Immediate

Table F.2 — Parser Output by Domain

Domain	Parser Input	Term Types Used	Provenance
Zig source	.zig files	keyword, name, type ref, operator	File hash + byte offset + Zig version
C headers	.h files	name, type ref, keyword	File hash + byte offset + library version

Domain	Parser Input	Term Types Used	Provenance
English text	.txt/.md files	kind (noun/verb/adj), name, operator	Document hash + position
JSON API docs	.json files	name, type ref, number	File hash + path + API version
Prolog rules	.pl files	predicate, rule, fact, variable	File hash + line number

Table F.3 — Domain Eating vs Fine-Tuning

Property	Domain Eating (KB)	Fine-Tuning (Float LLM)
Time to add domain	Minutes to hours (parse + validate)	Hours to days (training run)
Compute required	CPU (parsing)	GPU cluster (gradient updates)
Neural network modified	No	Yes
Provenance preserved	Yes (source, offset, version)	No (dissolved into weights)
Version control	Hard filter by version tag	Mixed into weight space
Reversible	Yes (remove facts from KB)	No (weights permanently changed)
Cross-domain queries	Automatic via shared predicates	Requires retraining on combined data
Consistency checking	Prolog contradiction detection	None

Appendix G: Prototype Specification

Table G.1 — Model Architecture

Parameter	Value
Total parameters	124M
Layers	12
d model	768
Attention heads	12
d head	64
d ff	3072
Vocabulary	Term-based (TermType enum + value index)
Weight format	i32 value + i16 remainder (6 bytes)

Parameter	Value
Accumulator	i64 (forward), i128 (overflow protection)
Activation functions	Integer lookup tables from MATH-2 cache
Normalization	Bit-shift at layer boundaries
Implementation	Zig, zero external dependencies

Table G.2 — Knowledge Base for Prototype

Property	Value
Source material	Zig standard library
Zig version	0.14
Estimated facts	~50,000
Fact types	function signatures, type definitions, module structure, error sets
Provenance	File hash + byte offset per fact
Rules	~200-500 Zig-specific validation rules
Storage estimate	~50 MB (facts + provenance + indices)

Table G.3 — Comparison Baseline

Property	Integer + Prolog (this architecture)	Float BF16 (baseline)
Weight format	i32 + i16 (6 bytes)	BF16 (2 bytes)
Training arithmetic	Integer only	Float
Knowledge base	Prolog with provenance	None
Tokenization	Term-based (typed)	BPE
Verification	Prolog at every step	None
Deterministic	Yes	No
Context	Persistent KB	Fixed token buffer

Table G.4 — Success Criteria Measurement

Criterion	Metric	Method	Pass Condition
1. Convergence	Training loss	Compare loss curves	Integer ≤ float at same step count
2. Long-tail accuracy	Rare API completion rate	Test on low-frequency stdlib functions	Integer+Prolog > float baseline
3. Hallucination reduction	Nonexistent function call rate	Count calls to functions not in stdlib	Integer+Prolog < float baseline

Criterion	Metric	Method	Pass Condition
4. Determinism	Output variance across runs	Run inference 10 × on same input	Variance = 0 for integer
5. Session stability	Output quality at turn N	Evaluate at turn 1, 100, 1000	No degradation
6. Domain eating	Accuracy on new library	Add library facts, test completions	Correct without retraining

Table G.5 — Falsification Tests

ID	Claim Tested	Test	Falsified If
F1	Integer training converges	Train 124M model, sweep thresholds	No threshold produces convergence
F2	Prolog overhead acceptable	Measure generation time with/without Prolog	Prolog makes generation >10 × slower
F3	KB consistency holds at scale	Run 10,000+ turn session, check contradictions	Contradictions appear that wouldn't at turn 100
F4	Domain eating faster than fine-tuning	Time both processes on same domain data	Parsing + rules slower than fine-tuning
F5	Verification reduces hallucination	Compare hallucination rates with/without Prolog	No reduction in hallucination rate
F6	Term tokenization viable	Compare perplexity: Term-based vs BPE	Term-based worse at same model size

---

Appendix H: Implementation File Map

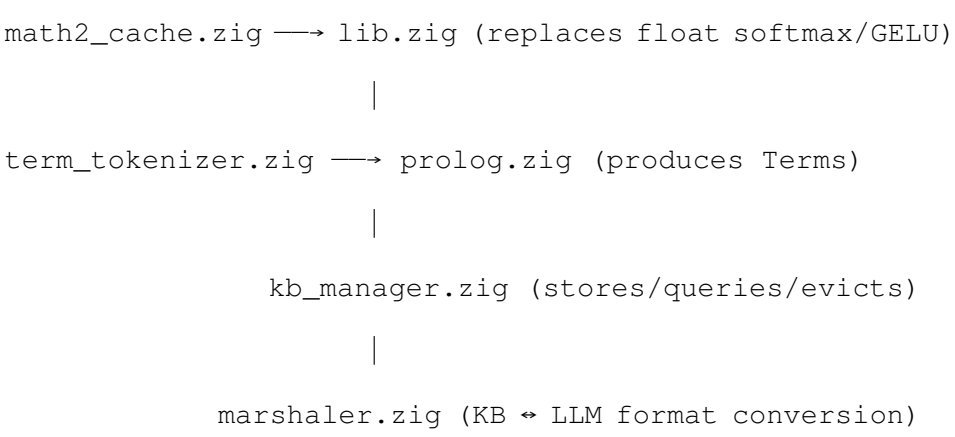
Table H.1 — Existing Components

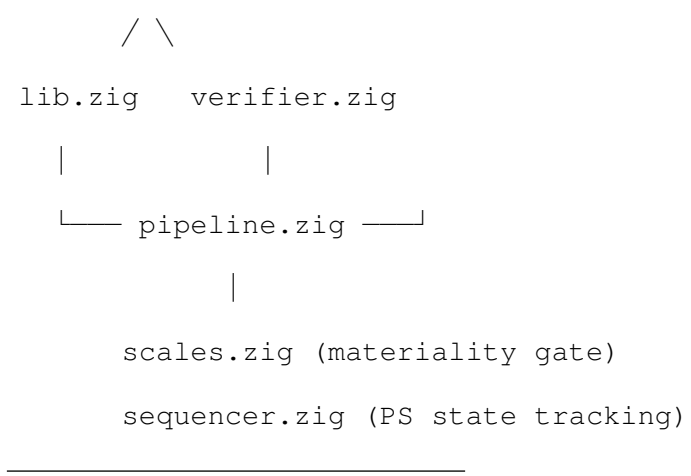
File	Status	Function	Modifications for Integration
lib.zig	Working	Weight struct, matmul, forward pass, backward pass, softmax, cross-entropy, weight init, checkpoint I/O, BPE tokenizer	Replace BPE with Term tokenizer. Replace f32 in softmax with MATH-2 lookup. Add Prolog marshaling interface.
tokenize.zig	Working	BPE training on source text	Replace with Term-based parser using Zig's own tokenizer
train.zig	Working	Training loop with shell transition monitoring	Add Prolog fact ingestion. Add KB update after each verified generation.
infer.zig	Working	Autoregressive greedy decoding	Add Prolog verification loop. Add KB query for context. Add retry on rejection.
eval.zig	Working	Generate code, attempt compilation, report pass/fail	Add hallucination detection (calls to nonexistent functions). Add provenance reporting.
prolog.zig	Working (game engine)	Term, Fact, Rule, FactSet, RuleSet, KnowledgeBase	Add provenance fields to Term. Add version filtering. Add contradiction detection. Add LRU eviction. Replace f32 with i32 in number terms.

Table H.2 — New Components Required

Component	Function	Estimated Size	Dependencies
term tokenizer.zig	Parse Zig source to typed Terms with provenance	~500 lines	Zig std tokenizer, prolog.zig
kb manager.zig	KB operations: assert, query, supersede, contradict, evict	~800 lines	prolog.zig
marshaler.zig	Convert KB facts to LLM input format and LLM output to Terms	~400 lines	prolog.zig, lib.zig
verifier.zig	Prolog verification of LLM-generated Terms against KB	~600 lines	prolog.zig, kb manager.zig
pipeline.zig	Alternating LLM↔Prolog generation loop with retry	~400 lines	All above
scales.zig	Materiality gating (Scales Method)	~200 lines	prolog.zig
sequencer.zig	Pseudo-Socratic state tracking and step selection	~400 lines	prolog.zig, kb manager.zig
math2 cache.zig	MATH-2 integer pair lookup tables for transcendentals	~300 lines	None (self-contained)

Table H.3 — Integration Dependencies





Status: Architecture specification with working components

Implementation: Zig, zero floats, zero external dependencies

Neural network: i32 weights, i16 remainders, integer matmul, integer backprop

Knowledge base: Typed Terms with provenance, version filtering, contradiction detection

Evaluation: Quadrium (L/M/E/U) on every fact, all integer

Controllers: Scales Method (materiality), Pseudo-Socratic (sequencing), Information Locality (trust)

Pipeline: LLM → Prolog → LLM, bidirectional, verified at every step

Context window: Replaced by persistent provenanced KB

Determinism: Guaranteed — integer arithmetic is associative

Domain eating: Parser + rules, no retraining

Prototype: 124M params, Zig stdlib, code completion, compared to float baseline

[**HOWL-LLM-1-2026**] Integer LLM with Prolog Knowledge Base. Github: https://github.com/ghowland/papers/tree/main/papers/LLM/HOWL-LLM-1-2026

[**HOWL-MATH-2-2026**] Integer-Pair Representations of Transcendental Constants at Sub-Planck Precision. Github: https://github.com/ghowland/papers/tree/main/papers/MATH/HOWL-MATH-2-2026

[**HOWL-INFO-1-2026**] Multi-Dimensional Information Indexing. Github: https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-1-2026

[**HOWL-SOPH-2-2026**] The Quadrium. Github: https://github.com/ghowland/papers/tree/main/papers/SOPH/HOWL-SOPH-2-2026

[**HOWL-INFO-2-2026**] The Scales Method. Github: https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL
INFO-2-2026

[**HOWL-INFO-3-2026**] The Pseudo-Socratic Method. Github: https://github.com/ghowland/papers/tree/main/papers/INF
INFO-3-2026

[**HOWL-INFO-4-2026**] Howland's Axiom of Information Locality. Github:
https://github.com/ghowland/papers/tree/main/papers/INFO/HOWL-INFO-4-2026